

3. The "Gotcha" (Risk Mitigation)

- **The "Profit Leak":** If a user buys a 10 sub and immediately spends 9 on Sonnet 5 (if you accidentally left it enabled for them), your 70% margin is destroyed.
 - **Solution:** Use **Model Routing/Permissions**. Create a whitelist of allowed models for each subscription tier. The Lightweight tier's API key or logic should simply *not* accept requests for Sonnet 5.
- **Latency:** Because you are acting as a proxy, your server adds ~50–100ms of latency. This is negligible for text-based apps but critical for streaming. Ensure you use streaming (SSE) in your backend to keep the UI responsive.
- **Credit Expiry:** To ensure financial stability, define if credits roll over. If they don't roll over, your margin increases significantly for low-usage users.

4. Recommended Tech Stack

- **Database:** PostgreSQL (to track user_id, creditsbalance, subscriptiontier, and usagelogs).
- **Backend:** Node.js or Python (FastAPI). Use a library like LiteLLM to handle the abstraction of different model providers behind a single standard API format. LiteLLM
- **Monitoring:** Use Helicone or LangSmith. These tools will track your costs in real-time, allowing you to see if your 70/30 margin is holding up across your entire user base. Helicone

Recommendation: Do not build a "fixed access" system. Build a "prepaid credit" system. It is the only way to guarantee your 70% margin regardless of which models the user chooses within their tier.

[illegible]

request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops." When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops."

When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops." When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops."

When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops." When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops."

When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops." When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.Hard Limits: Implement "Hard Stops."

When a user hits their 5-hour limit or runs out of credits, the backend rejects the request immediately. This prevents "over-consumption" from eating your profit margin.

asd

asd

asd

asd